ТОО ”Колледж Хекслет”

Лабораторная работа №6

CRUD-операции с использованием Prisma

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. CRUD-операции с использованием Prisma.

Цель: Реализовать операции создания, чтения, изменения и удаления товаров через Prisma Client, сохранив существующую архитектуру Express-приложения и REST API.

Условие: RO4_3/lesson6/ЛР-6.pdf

Требования задания

• В Product Service отсутствует pool.query().

• GET /api/products использует findMany().

• GET /api/products/:id использует findUnique().

• POST /api/products использует create().

• PATCH /api/products/:id использует update().

• DELETE /api/products/:id использует delete().

• POST возвращает статус 201.

• Несуществующий Product приводит к 404.

• Validation middleware продолжает использоваться.

• Routes не содержат Prisma-запросов.

• Controllers не содержат Prisma-запросов.

• Все изменения сохраняются в PostgreSQL.

Ход выполнения

Product Service использует только Prisma Client: findMany, findUnique, create, update и delete. Разрешённые поля явно отбираются. P2025 преобразуется в отсутствие записи и HTTP 404, ошибка внешнего ключа — в 409. Routes и Controller не содержат Prisma-запросов.

Проект: RO4_3/lesson6

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

services/productService.js

services/productService.js

```
import { prisma } from '../config/prisma.js';
export async function getAllProducts(query = {}) {
    const where = {};
    if (query.category) where.category = { name: query.category };
    if (query.inStock !== undefined) where.inStock = query.inStock === 'true';
    if (query.minPrice !== undefined || query.maxPrice !== undefined) {
        where.price = {};
        if (query.minPrice !== undefined) where.price.gte = Number(query.minPrice);
        if (query.maxPrice !== undefined) where.price.lte = Number(query.maxPrice);
    }
    return prisma.product.findMany({ where, orderBy: { id: "asc" } });
}
export function findProductById(id) { return prisma.product.findUnique({ where: { id } }); }
function productData(input) {
    return Object.fromEntries(['name', 'price', 'categoryId', 'inStock'].filter(key => key in input).map(key => [key, input[key]]));
}
export function addProduct(data) { return prisma.product.create({ data: productData(data) }); }
export async function updateProductById(id, changes) {
    try { return await prisma.product.update({ where: { id }, data: productData(changes) }); }
    catch (error) { if (error.code === 'P2025') return null; throw error; }
}
export async function deleteProductById(id) {
    try { return await prisma.product.delete({ where: { id } }); }
    catch (error) { if (error.code === 'P2025') return null; throw error; }
}
```

Листинг 1. services/productService.js

routes/productRoutes.js

routes/productRoutes.js

```
import { validateProductQuery } from '../middleware/validateProductQuery.js';
import { Router } from 'express';
import * as controller from '../controllers/productController.js';
import { validateId } from '../middleware/validateId.js';
import checkApiKey from '../middleware/checkApiKey.js';
import { validateCreateProduct } from '../middleware/validateCreateProduct.js';
import { validateUpdateProduct } from '../middleware/validateUpdateProduct.js';
const router = Router();
router.param('id', validateId);
router.use((req, res, next) => { console.log('Product Router:', req.method, req.originalUrl); next(); });
router.get('/', validateProductQuery, controller.getProducts);
router.get('/:id', controller.getProductById);
router.post('/', checkApiKey, validateCreateProduct, controller.createProduct);
router.patch('/:id', validateUpdateProduct, controller.updateProduct);
router.delete('/:id', controller.deleteProduct);
router.patch('/:id/settings', controller.productSettings);
export default router;
```

Листинг 2. routes/productRoutes.js

controllers/productController.js

controllers/productController.js

```
import * as service from '../services/productService.js';
import { AppError } from '../errors/AppError.js';

export async function getProducts(req, res, next) {
    try { res.json(await service.getAllProducts(req.query)); }
    catch (error) { next(error); }
}
export async function getProductById(req, res, next) {
    try {
        const product = await service.findProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export async function createProduct(req, res, next) {
    try { res.status(201).json(await service.addProduct(req.body)); }
    catch (error) { next(error); }
}
export async function updateProduct(req, res, next) {
    try {
        const product = await service.updateProductById(Number(req.params.id),
req.body);
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export async function deleteProduct(req, res, next) {
    try {
        const product = await service.deleteProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export function productSettings(req, res) {
    res.json({ id: Number(req.params.id), notify: req.query.notify === 'true', changes:
```

Листинг 3. controllers/productController.js (часть 1)

controllers/productController.js

```
req.body });
}
```

Листинг 4. controllers/productController.js (часть 2)

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1/items → 200

GET /api/orders/999999/items → 404

GET /api/orders/abc/items → 400

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "categoryId": 1,
  "inStock": true
}
```

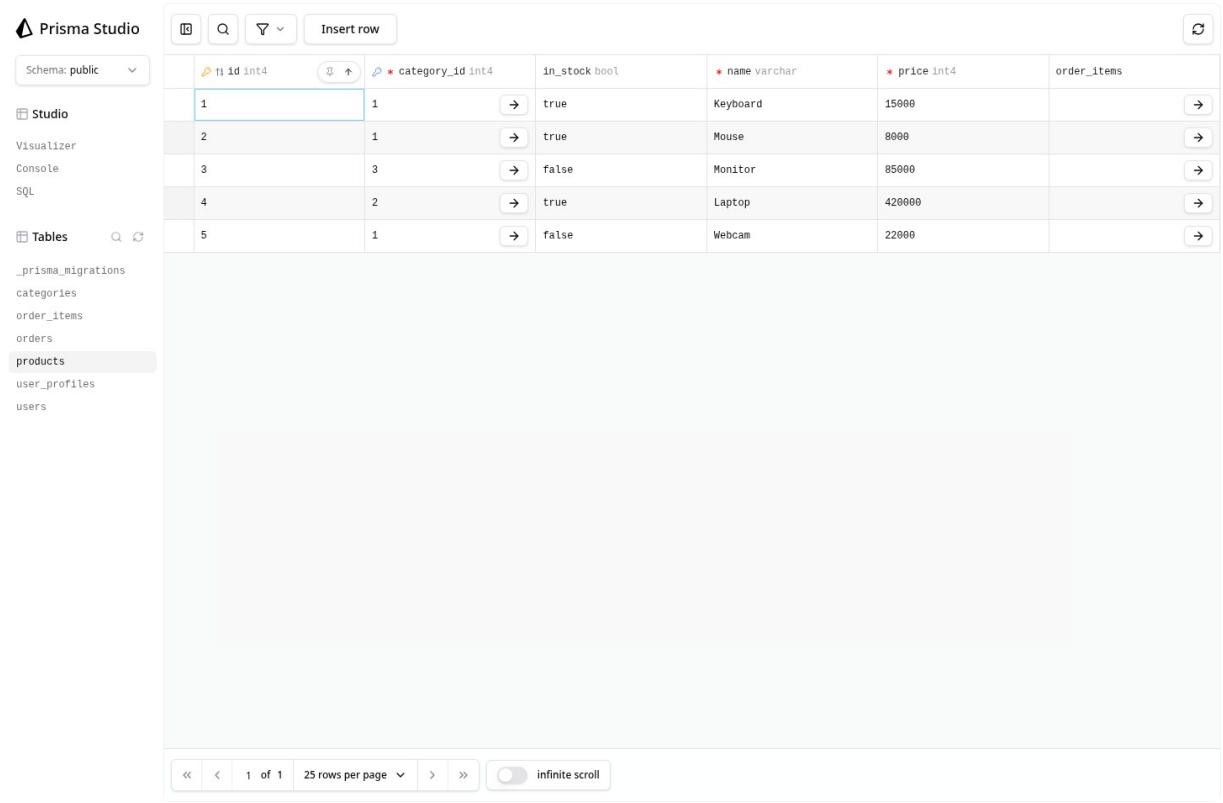

PATCH /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "categoryId": 1,
  "inStock": false
}
```


DELETE /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "categoryId": 1,
  "inStock": false
}
```

Проверка данных в Prisma Studio



Prisma Studio: таблицы и данные проверенной учебной базы.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.